

Big Data Analytics Programming

Solutions

Luca Stradiotti luca.stradiotti@kuleuven.be
Maaïke Van Roy maaike.vanroy@kuleuven.be

2025-2026

Exercise Session: Bits, Bytes, Instructions, and SIMD

In this session, we will be looking at the CPU and how it operates on (big) data using instructions. *Single Instruction, Multiple Data (SIMD)*, a set of instructions that allows one operation to be applied to multiple data items at once, is particularly important to big data applications, as it can considerably affect processing throughput. We will focus on the x86-64 instruction set, which is implemented by Intel's and AMD's server, desktop, and laptop processors.

Some parts of this exercise session are based on the excellent guide available from <https://web.stanford.edu/class/archive/cs/cs107/cs107.1202/guide/x86-64.html>.

1 CPU and Main Memory

The CPU or central processing unit is the computational core of your computer, and is accompanied by main memory to perform its tasks. The CPU runs a fetch, decode, and execute loop (the *Von Neumann model*):

- **fetch:** A *program counter* (the `%rip` register) stores the address of the instruction to be executed next. The fetch step loads the necessary bytes describing the instruction from memory.
- **decode:** The instruction that was just fetched from memory is interpreted by the CPU. The CPU distinguishes between the different instructions by looking at the *opcode* of the instruction. Besides the opcode, an instruction also contains information about the *operands*.
- **execute:** After having decoded the instruction and its operands, the different components of the CPU are activated. Different things happen for different instructions:
 - Data transfer instructions move data between main memory and the registers of the CPU.
 - Arithmetic or logical instructions take two pieces of data and compute a result (e.g. add, subtract, logical or, comparisons, etc.). A distinction is made between operations performed on integers and floating point numbers.
 - Jump instructions allow (conditional) jumps through the program. For other instructions, the program counter is incremented after each execution of an instruction. Jump instructions modify the program counter in other useful ways, allowing conditional statements, loops, function calls, etc.
- **memory and register write back** At the end of the cycle, the results of the operation are stored, either in memory or in a register.

Memory and the CPU are in constant interaction. However, the CPU is clocked higher than main memory and accessing main memory has considerable latency, so very often, the CPU will have to wait.

This issue is resolved by having caches. Caches temporarily store parts of memory closer to the CPU, allowing much faster access. When optimizing code, thinking about cache access patterns is crucial.

In reality, the fetch, decode, and execute cycle is a lot more involved. First of all, modern CPUs implement *pipelining*, which interleaves the cycles of multiple consecutive instructions. This can cause issues for *branching* instructions (i.e., jump instructions), as the CPU does not know which instruction is going to be next until the results of the comparison are known. Modern CPUs try to mediate this issue by doing *branch prediction*. Another reason why the instruction cycle is more involved is because in x86-64, instructions are not elementary operations; they are themselves made up of micro-operations. This means that some instructions can take longer to execute than others, which has considerable effects on dealing with dependences between computed values.

1.1 Memory Layout: Big and Little Endianness

The order in which bytes are stored in memory depends on the CPU architecture. The x86-64 architecture uses little endian, which places the least significant byte of a multi-byte number first (i.e. in a memory cell with a lower address). Big endian does the opposite. You can illustrate this with the following program (remember: `uint8_t *ptr` means `ptr` is a pointer to an unsigned 8-bit integer, the ampersand operator computes the address of a value):

code/session2/endianness.c

```
#include <stdint.h>
#include <stdio.h>

int main()
{
    uint16_t large_number = 0x0301; // 00000011 00000001 in binary
    uint8_t *ptr = (uint8_t *) &large_number;

    printf("first byte = 0x%x, second byte = 0x%x\n", ptr[0], ptr[1]);
}
```

```
% gcc endianness.c && ./a.out
first byte = 0x1, second byte = 0x3
```

Question 1. Does this experiment confirm the endianness of x86-64? What would the output be for big endian?

Solution 1 Yes. For big endian, the output would be 0x3, 0x1. ◇

1.2 Memory Alignment

When loading a 64-bit or 8-byte integer from memory, the pointer to this memory should align with an 8-byte boundary, or in other words, it must be a multiple of 8. This is true in general: unless you have a specialized instruction that can handle un-aligned memory access, all *b*-byte memory accesses should be *b*-byte aligned. If not, your program might be very slow, or might even crash.

The compiler will actually automatically add padding bits to structs to ensure memory alignment:

```
#include <stdint.h>
#include <stdio.h>
struct S { uint8_t c; uint32_t b; };
int main()
{
    struct S s;
    printf("s.c at 0x%lx, s.b at 0x%lx, diff=%ld\n",
        (uint64_t) &s.c, (uint64_t) &s.b,
        (uint64_t) &s.b - (uint64_t) &s.c);
}
```

```
% gcc aligned_struct.c && ./a.out
s.c at 0x4defe6a8, s.b at 0x4defe6ac, diff=4
```

Question 2. What value for diff would you expect if the S struct was not padded?

Solution 2 An uint8_t takes a single byte, so the answer would have been 1, which would have made s.b incorrectly aligned. ◇

2 An x86-64 Overview

2.1 Registers

The 32-bit registers names refer to the lowermost 32 bits of the 64-bit registers. You can also access the lowermost 16 bits (ax, bx, r8w, ...) and 8 bits (al, dil, r8b, ...), but we won't be using this here.

64-bit	32-bit	Historical use	Current use
rax	eax	Accumulator	Return register
rbx	ebx	Base	Callee-saved
rcx	ecx	Counter	4th argument
rdx	edx	Data	3rd argument
rsp	esp	Stack pointer	Stack pointer
rbp	ebp	Stack base pointer	Opt. frame pointer
rsi	esi	Source	2nd argument
rdi	edi	Destination	1st argument
r8-r15	r8-r15d	General purpose	Mixed use

Question 3. How could you verify ebx populates the lowermost 32 bits of rbx?

Solution 3 Write a value in ebx. Check if it affects the lower half of rbx. ◇

2.2 Addressing Modes

Operands of instructions may be values, registers, pointers to values in memory, and pointers to pointers to values in memory. The following table gives an overview using the mov instruction.¹

¹We'll use the AT&T assembler syntax: https://en.wikibooks.org/wiki/X86_Assembly/GAS_Syntax. An alternative is the Intel syntax.

Instruction	Explanation
<code>mov \$1, %eax</code>	Move (immediate) value 1 to register <code>eax</code>
<code>mov %edi, %eax</code>	Move value of register <code>edi</code> to register <code>eax</code>
<code>mov \$1, (%eax)</code>	Move value 1 to to address in memory stored in <code>eax</code>
<code>mov (%edi), %eax</code>	Move contents in memory stored at address in <code>edi</code> to register <code>eax</code>
<code>mov \$1, -4(%ebp)</code>	Store value 1 in stack variable; address is computed as <code>ebp - 4</code> ; -4 is called the <i>displacement</i> and <code>ebp</code> is the <i>base address</i>
<code>mov \$1, 8(%esp, %edi, 4)</code>	Store value 1 in <i>address</i> ; <i>address</i> is <code>esp + 4*edi + 8</code> ; 4 is called the scaling factor (e.g., the width of the elements in an array)
<code>mov \$1, 0x10(, %edx, 8)</code>	Store value 1 in <i>address</i> ; <i>address</i> is <code>0 + 8*edx + 16</code> ; this is a special case of the previous with base 0 assumed

All cases where registers are in parenthesis are called *indirect addressing*.

Sometimes, suffixes are added to the instruction mnemonics to indicate the size of the operands. The following table gives an overview of the suffixes:

Suffix	Meaning
b	byte (1 byte)
w	word (2 bytes)
l	long (4 bytes)
q	quad (8 bytes)
s	single precision float (4 bytes)
l	double precision float (8 bytes)

The `lea` instruction can be used to compute addresses. For example `'lea 0x20(%rsp), %rdi'` stores `rsp + 32` in `rdi`. In other words, `lea` is simply an arithmetic operation; and for this reason, you will often see it used for simple arithmetic operations on values that are not necessarily memory addresses.

2.3 Stack Frame Layout and Calling Conventions

We will be focusing on the System V ABI², used by most compilers and operating systems except Microsoft Windows. We will briefly cover how arguments are passed from caller to callee and how the stack is used to store local variables and optionally arguments.

- **The stack frame:** Two registers `rbp` (base pointer) and `rsp` (stack pointer) are used to maintain pointers to fixed parts in the current function's stack frame. `rbp` points to its own previous value (so we can restore the stack frame of the calling function). Above `rbp`, the return address and the arguments passed by memory can be found. Below `rbp`, the local variables are stored. Functions that don't call functions themselves can use the red zone for scratch without having to update `rsp`.

²Application Binary Interface, like an API, but then in binary, compiled format. See <https://github.com/hjl-tools/x86-psABI/wiki/X86-psABI> for more information.

	Position	Contents	Frame
high address	8n + 16(%rbp)	memory argument <i>n</i> ...	previous
	16(%rbp)	memory argument 0	
low address	8(%rbp)	return address	current
	0(%rbp)	previous rbp value	
	-8(%rbp)	variable zone	
	0(%rsp)		
	-128(%rsp)		

- **Passing arguments and return values:** The first 6 (integer or pointer type) arguments that fit in 64-bit registers are passed using registers `rdi`, `rsi`, `rdx`, `rcx`, `r8` and `r9`. The remaining arguments are passed on the stack.

The return value is stored in `rax` or on the stack in memory if too large.

The above explanation is simplified, and in reality more complex situations can arise. See System V ABI documentation for all the details.

3 Common Instructions and Registers: An Example

The following example implements a program that takes an array length and two arrays as input, and computes the element-wise sum of this array, storing the result in the second array:

code/session2/add_ints.c

```
#include <stdio.h>
#define N 4

void __attribute__((noinline))
add_ints(int n, int *as, const int *bs)
{
    for (int i = 0; i < n; ++i)
        as[i] = as[i] + bs[i];
}

int main()
{
    int as[N] = {1, 2, 3, 4};
    int bs[N] = {1, -1, 5, -2};
    add_ints(N, &as[0], &bs[0]);

    for (int i = 0; i < N; ++i)
        printf("as[%d] = %d\n", i, as[i]);
    putchar('\n');
}
```

```
% gcc -g -O1 add_ints.c && ./a.out
bs[0] = 2   bs[1] = 1   bs[2] = 8   bs[3] = 2
```

Let's have a look at the instructions generated for this function by gcc:

```
gdb a.out -batch -ex 'disassemble add_ints'
Dump of assembler code for function add_ints:
0x00401136 <+0>:    test    %edi,%edi                ; *di contains n
0x00401138 <+2>:    jle     0x401154 <add_ints+30> ; return if *di <= 0
0x0040113a <+4>:    lea     -0x1(%rdi),%edi          ; *di = *di - 1
0x0040113d <+7>:    mov     $0x0,%eax               ; i = 0 (*ax)
```

```

0x00401142 <+12>:  mov    (%rdx,%rax,4),%ecx    ; tmp = as[i]
0x00401145 <+15>:  add    %ecx,(%rsi,%rax,4)    ; bs[i] += tmp
0x00401148 <+18>:  mov    %rax,%rcx            ; tmp = i
0x0040114b <+21>:  add    $0x1,%rax            ; i += 1
0x0040114f <+25>:  cmp    %rdi,%rcx            ; cmp(n-1, tmp)
0x00401152 <+28>:  jne    0x401142 <add_ints+12> ; loop again if i != n-1
0x00401154 <+30>:  retq                       ; return to caller

```

End of assembler dump.

Depending on the optimization level, the compiler might generate very different code: -O3 will most likely apply loop-unrolling or it might even try to vectorize the loop.

Question 4. Try to identify what the different instructions of `sum_sq_ints` do:

code/session2/sum_sq_ints.c

```

#include <stdio.h>
#define N 8

int __attribute__((noinline))
sum_sq_ints(int n, const int *as)
{
    int result = 0;
    for (int i = 0; i < n; ++i)
        result += as[i] * as[i];
    return result;
}

int main()
{
    int as[N] = {1, 2, 3, 4, 5, 6, 7, 8};
    printf("result = %d\n", sum_sq_ints(N, &as[0]));
}

```

Solution 4 Compile the code and use gdb to look at the generated instructions:

```

gcc -g -O1 sum_sq_ints.c
gdb a.out -batch -ex 'disassemble sum_sq_ints'
Dump of assembler code for function sum_sq_ints:
0x00401126 <+0>:  test    %edi,%edi ; same as before
0x00401128 <+2>:  jle     0x40114d <sum_sq_ints+39>
0x0040112a <+4>:  mov     %rsi,%rdx    ; *dx = &as[0]
0x0040112d <+7>:  lea     -0x1(%rdi),%eax    ; *ax = n - 1
0x00401130 <+10>: lea     0x4(%rsi,%rax,4),%rsi ; *si = end of array
0x00401135 <+15>: mov     $0x0,%ecx    ; result = 0
0x0040113a <+20>: mov     (%rdx),%eax    ; tmp = as[0]
0x0040113c <+22>: imul    %eax,%eax    ; tmp *= tmp
0x0040113f <+25>: add     %eax,%ecx    ; result += tmp
0x00401141 <+27>: add     $0x4,%rdx    ; make *dx point to next el in as
0x00401145 <+31>: cmp     %rsi,%rdx    ; cmp *dx with end of array
0x00401148 <+34>: jne     0x40113a <sum_sq_ints+20> ; continue if not end
0x0040114a <+36>: mov     %ecx,%eax    ; return value = result
0x0040114c <+38>: retq                      ; return to main
0x0040114d <+39>: mov     $0x0,%ecx    ; result = 0
0x00401152 <+44>: jmp     0x40114a <sum_sq_ints+36>

```

End of assembler dump.

◇

4 SIMD Intrinsics

SIMD instructions or *Single Instruction, Multiple Data* instructions apply the same operation to multiple data elements at once. This can be very useful, as many programs loop over one or multiple arrays of data elements and apply a certain operation to these elements. If these operations can be applied to multiple elements at the same time rather than one by one, we obviously have a speed improvement.

The main difference between a regular instruction and an SIMD instruction is the fact they operate on specialized registers. We'll be focusing on the AVX and AVX2 extensions for x86-64, which define instructions that operate on the ymm1 to ymm15 registers. `__m256i` is a 32-byte integer type. Here's an example:

code/session2/ymm_regs.c

```
#include <stdio.h>
#include <immintrin.h>
#include "print_bits.c"

int main()
{
    __m256i a, b, c;
    int as[8] = {0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1, -0x12345678};

    a = _mm256_set_epi32(0x12345678, 0x7, 0x6, 0x5, 0x4, 0x3, 0x2, 0x1);
    b = _mm256_loadu_si256((__m256i *) &as[0]);
    c = _mm256_add_epi32(a, b);

    printf("a = "); print_hex(&a, 256);
    printf("b = "); print_hex(&b, 256);
    printf("c = "); print_hex(&c, 256);
}
```

```
% gcc -O1 -g -mavx -mavx2 ymm_regs.c && ./a.out
a = 01000000 02000000 03000000 04000000 05000000 06000000 07000000 78563412
b = 07000000 06000000 05000000 04000000 03000000 02000000 01000000 88a9cbcd
c = 08000000 08000000 08000000 08000000 08000000 08000000 08000000 00000000
```

Question 5. Why is the last part of a mangled? What does c contain?

Solution 5 The number is stored in little endian in memory. This is also the reason why `0x01000000` is printed instead of `0x00000001`. c contains the element-wise sum of the 8 32-bit integers in a and the 8 32-bit integers in b. ◇

Question 6. Why do we use `_mm256_loadu_si256` instead of `_mm256_load_si256`?

Solution 6 The former allows non-32-byte-aligned loads. The alignment of as is 4 bytes and not necessarily 32 bytes. ◇

A second example re-implements the `add_ints` program from earlier using SIMD intrinsic functions:

```

#include <immintrin.h>

void __attribute__((noinline))
add_ints(int n, int *as, const int *bs)
{
    int i = 0, n8 = (n / 8) * 8;
    for (; i < n8; i += 8)
    {
        __m256i a = _mm256_loadu_si256((__m256i *) &as[i]);
        __m256i b = _mm256_loadu_si256((const __m256i *) &bs[i]);
        a = _mm256_add_epi32(a, b);
        _mm256_storeu_si256((__m256i *) &as[i], a);
    }

    for (; i < n; ++i)
        as[i] += bs[i];
}

```

Question 7. What do the two for loops do?

Solution 7 The first for loops over tuples of eight pairs of elements at once. The last loop takes the remaining pairs that do not fill up an 8-element register and sums them up one by one. ◇

Question 8. Use the Intel Intrinsics Guide to vectorize the `add_ints` and `sum_sq_ints` functions using AVX and AVX2 intrinsic functions. Look at the generated instructions. Can you identify the instructions that work on the wide registers?

You can use `gdb` to investigate the effects of these instructions. Example usage:

```

% gdb a.out
break sum_sq_ints ; set a breakpoint at sum_sq_ints
layout split      ; show code and instructions
run              ; start program execution
step / next      ; step one line code (skip over functions)
stepi            ; step one instruction
continue / finish ; continue until breakpoint (or end of function)
print $xmm0       ; print value stored in register xmm0
display $xmm0     ; track value stored in register xmm0

```

The generated instructions may use the `xmm*` registers. The `xmm*` registers are aliases of the low half of the respective `ymm*` registers. (Just like `eax` is an alias of the low half of `rax`.)

Solution 8 Code for `sum_sq_ints`:

code/session2/solution/sum_sq_ints_vect.c

```

#include <immintrin.h>

int __attribute__((noinline))
sum_sq_ints(int n, const int *as)
{
    __m256i result = _mm256_setzero_si256();
    int i = 0, n8 = (n / 8) * 8;
    for (; i < n8; i += 8)
    {
        __m256i a = _mm256_loadu_si256((__m256i *) &as[i]);
        a = _mm256_mullo_epi32(a, a);
        result = _mm256_add_epi32(a, result);
    }

    int r = 0;
    for (; i < n; ++i)
        r += as[i] * as[i];

    result = _mm256_hadd_epi32(result, result);
    result = _mm256_hadd_epi32(result, result);
    r += _mm256_extract_epi32(result, 0);
    r += _mm256_extract_epi32(result, 4);

    // you could also just call _mm256_extract_epi32 8 times!

```

```

% gcc -O1 -mavx -mavx2 sum_sq_ints_vect.c
% gdb a.out -batch -ex 'disassemble sum_sq_ints'
Dump of assembler code for function sum_sq_ints:
0x00401126 <+0>: lea    0x7(%rdi),%eax    ; *ax = n + 7 (*di contains n arg)
0x00401129 <+3>: test   %edi,%edi
0x0040112b <+5>: cmovns %edi,%eax        ; *ax = n if n >= 0 else n + 7
0x0040112e <+8>: and    $0xffffffff8,%eax ; *ax = (*ax / 8) * 8 (zero 3 LSBs)
0x00401131 <+11>: mov    %eax,%edx        ; copy *ax to *dx
0x00401133 <+13>: jle    0x4011bf <sum_sq_ints+153> ; jump if n < 7
0x00401139 <+19>: mov    %rsi,%rax        ; *ax = &as[0]
0x0040113c <+22>: sub    $0x1,%edx        ; *dx -= 1 (== (n/8)*8 - 1)
0x0040113f <+25>: shr    $0x3,%edx        ; *dx /= 8
0x00401142 <+28>: mov    %edx,%ecx        ; copy *dx to *cx
0x00401144 <+30>: shl    $0x5,%rcx        ; *cx *= 32
0x00401148 <+34>: lea    0x20(%rsi,%rcx,1),%rcx ; *cx stores end as array mod 8
0x0040114d <+39>: vpxor  %xmm0,%xmm0,%xmm0 ; zero result vector
0x00401151 <+43>: vmovdqu (%rax),%xmm2    ; fill ymm1 with first 4 values ...
0x00401155 <+47>: vinsertri128 $0x1,0x10(%rax),%ymm2,%ymm1 ; ... and last 4 values
0x0040115c <+54>: vpmulld %ymm1,%ymm1,%ymm1 ; compute elem-wise square
0x00401161 <+59>: vpaddd  %ymm0,%ymm1,%ymm0 ; elem-wise add to result
0x00401165 <+63>: add     $0x20,%rax       ; move to next 8 elements
0x00401169 <+67>: cmp     %rcx,%rax        ; check if end of array mod 8 reached
0x0040116c <+70>: jne     0x401151 <sum_sq_ints+43> ; repeat if not at end (== for1)
0x0040116e <+72>: lea     0x8(,%rdx,8),%eax ; *ax = *dx * 8 + 8
0x00401175 <+79>: cmp     %eax,%edi        ; check if elements remaining (edi=n)
0x00401177 <+81>: jle     0x4011ca <sum_sq_ints+164> ; jump is no elements remaining
0x00401179 <+83>: movslq  %eax,%rcx        ; 32-bit -> 64-bit sign extend
0x0040117c <+86>: lea     (%rsi,%rcx,4),%rdx ; *dx = &as[*cx]; *cx is next index
0x00401180 <+90>: sub     $0x1,%edi        ; *di = n - 1
0x00401183 <+93>: sub     %eax,%edi        ; how many elements remaining? - 1
0x00401185 <+95>: add     %rcx,%rdi        ; index of last element
0x00401188 <+98>: lea     0x4(%rsi,%rdi,4),%rsi ; *si is end of array
0x0040118d <+103>: mov     $0x0,%ecx        ; *cx = r = 0
0x00401192 <+108>: mov     (%rdx),%eax      ; fetch single elements from as
0x00401194 <+110>: imul    %eax,%eax        ; square
0x00401197 <+113>: add     %eax,%ecx        ; add to r
0x00401199 <+115>: add     $0x4,%rdx        ; advance pointer
0x0040119d <+119>: cmp     %rsi,%rdx        ; check if end of as reached

```

```

0x004011a0 <+122>: jne      0x401192 <sum_sq_ints+108> ; repeat if not end
0x004011a2 <+124>: vphadd %ymm0,%ymm0,%ymm0 ; horizontal add -> ymm0[0]=sum 0..4
0x004011a7 <+129>: vphadd %ymm0,%ymm0,%ymm0 ; horizontal add -> ymm0[4]=sum 4..8
0x004011ac <+134>: vmovd   %xmm0,%eax          ; get xmm0[0] == ymm[0]
0x004011b0 <+138>: add     %eax,%ecx          ; add to r
0x004011b2 <+140>: vextracti128 $0x1,%ymm0,%xmm0 ; move high half to low half ymm0
0x004011b8 <+146>: vmovd   %xmm0,%eax          ; get xmm0[0] (was ymm[4])
0x004011bc <+150>: add     %ecx,%eax          ; add to r, store in return value reg
0x004011be <+152>: retq                                ; return
0x004011bf <+153>: mov     $0x0,%eax          ; zero *ax
0x004011c4 <+158>: vpxor   %xmm0,%xmm0,%xmm0 ; zero xmm0
0x004011c8 <+162>: jmp     0x401175 <sum_sq_ints+79>
0x004011ca <+164>: mov     $0x0,%ecx          ; *cx = r = 0
0x004011cf <+169>: jmp     0x4011a2 <sum_sq_ints+124>
End of assembler dump.

```

If you're interested in a description of what each instruction does, here's a reference: <https://www.felixcloutier.com/x86/>. ◇

5 Working With Individual Bits

Important data structures work on the bit level . To implement these structures, we need to work with individual bits. The following program implements a simple bitset:

code/session2/bitset.c

```

#include <stdint.h>
#include <stdio.h>
#include "print_bits.c"

int get_bit(int bit_index, uint8_t *bits)
{ int i = bit_index / 8, j = bit_index % 8; return (bits[i] >> j) & 0x1 == 0x1; }

void set_bit(int bit_index, uint8_t *bits)
{ int i = bit_index / 8, j = bit_index % 8; bits[i] |= 0x1 << j; }

void unset_bit(int bit_index, uint8_t *bits)
{ /* TODO implement */ }

int main()
{
    uint8_t a[32] = { 0 }, b[32] = { 0 }, c[32] = { 0 };
    for (int i = 0; i < 256; ++i)
    { if (i % 3 == 0) set_bit(i, &a[0]);
      if (i % 9 == 0) set_bit(i, &b[0]); }
    unset_bit(9, &a[0]);

    print_bits(&a[0], 64); print_bits(&b[0], 64); // print first 64 bits

    for (int i = 0; i < 256; ++i) // compute set intersection
    { int a_set = get_bit(i, &a[0]),
        b_set = get_bit(i, &b[0]);
      if (a_set && b_set) set_bit(i, &c[0]); }

    puts("\nset intersection:");
    print_bits(&c[0], 64);
}

```

```

% gcc -g bitset.c && ./a.out
0: 10010010 01001001 00100100 10010010 01001001 00100100 10010010 01001001
0: 10000000 01000000 00100000 00010000 00001000 00000100 00000010 00000001

set intersection:
0: 10000000 01000000 00100000 00010000 00001000 00000100 00000010 00000001

```

Question 9. Implement `unset_bit`.

Solution 9 An `unset_bit` implementation:

`unset_bit`

```
void unset_bit(int bit_index, uint8_t *bits)
{
    int i = bit_index / 8, j = bit_index % 8;
    bits[i] &= ~(0x1 << j);
}
```

◇

Question 10. How could you implement the set intersection more efficiently? Think about how you could use SIMD instructions to your advantage.

Solution 10 An SIMD set intersection implementation:

`bitset_and`

```
void bitset_and(int n, uint8_t *a, uint8_t *b, uint8_t *c)
{
    int i = 0, n8 = (n / 32) * 32;
    for (; i < n8; i += 32)
    {
        __m256i x = _mm256_loadu_si256((__m256i *) &a[i]);
        __m256i y = _mm256_loadu_si256((__m256i *) &b[i]);
        __m256i z = _mm256_and_si256(x, y);
        _mm256_storeu_si256((__m256i *) &c[i], z);
    }
    for (; i < n; ++i)
        c[i] = a[i] & b[i];
}
```

◇

Question 11 (extra). Compute the number of elements in the intersection set as quickly as possible.

Solution 11 To compute the size of the set, you need to compute the number of one bits. This operation is called the population count or *popcount*.

This repository hosts the most efficient popcount implementations: <https://github.com/WojciechMula/sse-popcount>.

◇

6 Auto-Vectorization by Compilers

All modern compilers will look for code templates and replace sequential code by equivalent vectorized code. In fact, all manual implementations from this exercise session would automatically be vectorized by the compiler. Even population counts are replaced. For more information: <https://gcc.gnu.org/projects/tree-ssa/vectorization.html> for GCC, and <https://llvm.org/docs/Vectorizers.html> for Clang.

In practice, only manually write SIMD code using intrinsics when (1) the compiler does not do it automatically, or it does it poorly, and (2), the program you are developing would benefit from the added performance. These two requirements are often true in big data systems like databases!

7 SIMD in Other Instruction Sets

The x86-64 instruction set is not the only instruction set with SIMD capabilities:

- ARM SIMD ISAs³: multiple instruction set extensions have been proposed, e.g. SVE and Neon.

³<https://developer.arm.com/architectures/instruction-sets/simd-isas>

(You can use Neon SIMD instructions on your Raspberry Pi!)

- MIPS SIMD⁴ and others. There have been multiple SIMD extensions for MIPS over the years, usually focusing on a specific applications like media and 3D graphics.
- Power ISA, SPARC, proposals for RISC-V, ...

8 The Big Question: “Why Can’t I Use Python For My BDAP Project?”

This is a frequent question. I hope this exercise session has convinced you that Python simply does not provide you with enough fine-grained control over the program and the generated instructions when working on performance critical code.

An interesting note: you might have already used SIMD instructions from Python! Indeed, optimized numpy array operations are very well optimized. These numpy operations are not implemented in plain Python, however. Some of them are implemented in Cython, others call heavily optimized libraries like *BLAS⁵ and LAPACK.

⁴<https://www.mips.com/products/architectures/ase/simd/>

⁵A popular BLAS implementation, OpenBLAS, is 50% Fortran, 25% handwritten assembly (just like the compiler generated assembly we looked at today, but handcoded!), and 20% C.